

SMART EXHIBITION & TRADE FAIR MANAGEMENT SYSTEM

Internship Project Report

Submitted By

Karan Singh Adhikari

Bachelor of Computer Applications (BCA IV A)

Amrapali University

Submitted To

COMPANY : Aarsh AI

COORDINATOR : Swasti Mishra

Technology Stack

MERN Stack

- MongoDB
 - Express.js
 - React.js
 - Node.js
-

Duration

[July-August]

Date of Submission

August 2026

TABLE OF CONTENTS

S. No.	Section	Page No.
1	Cover Page	1
2	Introduction, Problem Statement & Objectives	3
3	Technology Stack, System Architecture & Project Structure	5
4	Features, User Roles & System Workflow	7
5	Database Design, API Overview & Application Screenshots	9
6	Challenges Faced, Future Enhancements & Conclusion	10

INTRODUCTION

Project Overview

The **Smart Exhibition & Trade Fair Management System** is a web-based application developed to digitize and simplify the management of exhibitions and trade fairs. The system provides a centralized platform where organizers can manage events, exhibitors can reserve booths, and visitors can register for exhibitions through an intuitive user interface.

The application is built using the **MERN Stack (MongoDB, Express.js, React.js, and Node.js)** and follows a secure client-server architecture with role-based authentication.

Why This Project?

Traditional exhibition management is often performed manually using spreadsheets, paperwork, and multiple communication channels. As the number of events and participants grows, these methods become difficult to manage and prone to errors.

This project aims to automate these processes by providing a single platform for managing exhibitions efficiently.

Key Highlights

- Web-based exhibition management platform
 - Secure user authentication using JWT
 - Role-based access control
 - Online booth booking
 - Visitor registration system
 - QR code-based event tickets
 - Responsive and user-friendly interface
-

PROBLEM STATEMENT

Traditional exhibition management systems face several challenges:

- Manual booth allocation.
- Time-consuming registration process.
- Difficulty in managing exhibitor and visitor records.
- Lack of centralized event management.
- Increased chances of duplicate or inconsistent data.

OBJECTIVES & PROJECT SCOPE

Project Objectives

The primary objectives of the **Smart Exhibition & Trade Fair Management System** are:

- Develop a centralized platform for managing exhibitions and trade fairs.
 - Provide secure authentication with role-based access control.
 - Allow administrators to create, update, and manage exhibition events.
 - Enable exhibitors to browse and book available booths.
 - Allow visitors to register for exhibitions quickly and efficiently.
 - Generate QR-based digital tickets for registered visitors.
 - Maintain a centralized database for events, users, booths, and bookings.
 - Build a responsive and user-friendly interface using modern web technologies.
 - Apply MERN stack concepts in a real-world full-stack application.
-

Project Scope

The MVP (Minimum Viable Product) focuses on the core functionalities required to manage an exhibition successfully.

Admin

- Manage users.
- Create, edit, and delete exhibition events.
- Create and manage booths.
- Monitor bookings and visitor registrations.
- View overall system statistics through the dashboard.

Exhibitor

- Register and log in.
- Browse available exhibitions.
- View booth details.
- Book available booths.
- Track booking status from the dashboard.

Visitor

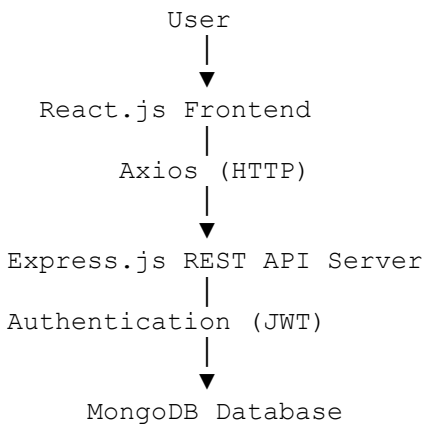
- Register and log in.
 - Browse upcoming exhibitions.
 - Register for an event.
 - Receive a QR-based digital entry ticket.
 - View registered events from the dashboard.
-

TECHNOLOGY STACK, SYSTEM ARCHITECTURE & PROJECT STRUCTURE

Technology Stack

Category	Technology	Purpose
Frontend	React.js	Building the user interface
Routing	React Router	Client-side navigation
State Management	Redux Toolkit / Context API	Managing global application state
Styling	Tailwind CSS	Responsive and modern UI design
HTTP Client	Axios	Communication between frontend and backend
Backend	Node.js	JavaScript runtime environment
Framework	Express.js	Building RESTful APIs
Database	MongoDB	Storing application data
ODM	Mongoose	Database modeling and validation
Authentication	JWT & bcrypt	Secure login and password encryption
Utilities	dotenv, cors, cookie-parser, express-validator	Environment configuration, CORS, cookies, and request validation
Additional Library	QR Code Generator	Generating QR-based entry tickets

System Architecture



Architecture Overview

- The **React** frontend provides an interactive user interface.
 - **Axios** sends HTTP requests to the backend.
 - **Express.js** processes business logic and API requests.
 - **JWT** secures protected routes and user authentication.
 - **MongoDB** stores users, events, booths, bookings, and visitor information.
-

Project Structure

Smart-Exhibition-Trade-Fair-Management-System/

```
├── client/
│   ├── src/
│   ├── public/
│   └── package.json
├── server/
│   ├── controllers/
│   ├── models/
│   ├── routes/
│   ├── middleware/
│   ├── config/
│   ├── utils/
│   └── server.js
├── documentation/
└── README.md
```

Folder Description

- **client/** – Contains the React frontend application.
 - **server/** – Contains the Express backend, APIs, and database logic.
 - **controllers/** – Handles business logic for API requests.
 - **models/** – Defines MongoDB schemas using Mongoose.
 - **routes/** – Contains all API route definitions.
 - **middleware/** – Authentication, authorization, and validation middleware.
 - **config/** – Database connection and environment configuration.
 - **documentation/** – Project report, screenshots, and supporting documents.
-

FEATURES, USER ROLES & SYSTEM WORKFLOW

Core Features

The Smart Exhibition & Trade Fair Management System provides the following core functionalities:

Authentication

- User Registration
- Secure Login & Logout
- JWT-based Authentication
- Role-Based Authorization

Event Management

- Create, update, and delete exhibitions
- View exhibition details
- Manage event information

Booth Management

- Create and manage booths
- View booth availability
- Book exhibition booths
- Track booking status

Visitor Registration

- Register for exhibitions
- Store visitor information
- Generate QR-based entry tickets

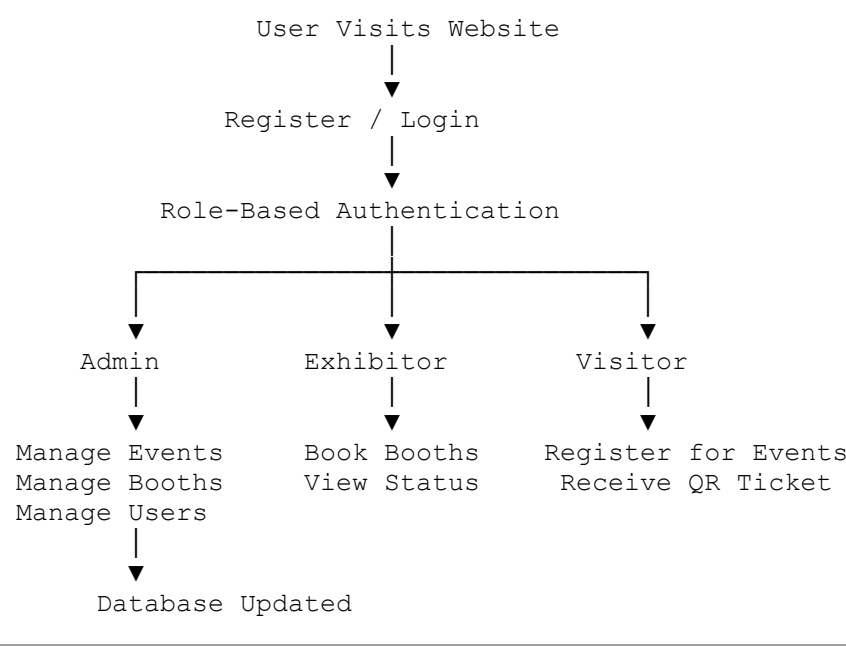
Dashboards

- **Admin Dashboard** – Manage events, booths, users, and bookings
 - **Exhibitor Dashboard** – View booked booths and booking status
 - **Visitor Dashboard** – View registered events and QR tickets
-

User Roles

Role	Responsibilities
Administrator	Manage users, exhibitions, booths, bookings, and monitor overall system activities.
Exhibitor	Browse exhibitions, reserve booths, and manage booth bookings.
Visitor	Register for exhibitions, receive QR tickets, and view registration history.

System Workflow



Key Benefits

- Centralized management of exhibitions and trade fairs.
- Secure access using role-based authentication.
- Faster booth booking process.
- Simplified visitor registration.
- Digital QR ticket generation.
- Responsive and user-friendly interface.
- Organized data management through MongoDB.

DATABASE DESIGN, API OVERVIEW & APPLICATION SCREENSHOTS

Database Design

The application uses **MongoDB** as its primary database. Data is organized into separate collections to ensure efficient storage, retrieval, and scalability.

Main Collections

Collection	Purpose
Users	Stores user information, authentication details, and roles (Admin, Exhibitor, Visitor).
Events	Stores exhibition details such as event name, description, venue, and schedule.
Booths	Stores booth information including booth number, size, price, availability, and associated event.
Bookings	Stores booth booking information, booking status, exhibitor details, and event references.
Visitors	Stores visitor registrations and generated QR ticket information.

API Overview

The backend follows a **RESTful API architecture**, where the frontend communicates with the server using HTTP requests.

Major API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Authenticate user
GET	/api/events	Retrieve all exhibitions
POST	/api/events	Create a new exhibition (<i>Admin</i>)
PUT	/api/events/:id	Update exhibition details
DELETE	/api/events/:id	Delete an exhibition
GET	/api/booths	Retrieve available booths
POST	/api/bookings	Create a booth booking
GET	/api/dashboard	Retrieve dashboard statistics

CHALLENGES FACED, FUTURE ENHANCEMENTS & CONCLUSION

Challenges Faced

During the development of the **Smart Exhibition & Trade Fair Management System**, several technical challenges were encountered. Overcoming these challenges helped improve the overall understanding of full-stack application development.

Major Challenges

- Designing a secure role-based authentication system using JWT.
 - Managing communication between the React frontend and Express backend.
 - Designing MongoDB schemas and maintaining relationships between collections.
 - Implementing booth booking while preventing duplicate reservations.
 - Managing application state efficiently across multiple user roles.
 - Creating responsive and user-friendly interfaces for different screen sizes.
 - Handling API validation, error responses, and exception handling.
-

Future Enhancements

The current application represents the Minimum Viable Product (MVP). Future versions can include several advanced features to improve functionality and user experience.

Planned Improvements

- Online payment gateway integration for booth bookings.
 - QR code scanning for event attendance.
 - Email and SMS notifications for registrations and bookings.
 - Cloudinary integration for event and booth image uploads.
 - Analytics dashboard with charts and reports.
 - Real-time notifications using Socket.io.
 - Feedback and review system.
 - Certificate generation for exhibitors and participants.
 - Sponsor management module.
 - Mobile application support.
-

Conclusion

The **Smart Exhibition & Trade Fair Management System** successfully demonstrates the implementation of a modern full-stack web application using the MERN stack. The project provides a centralized platform for managing exhibitions, booth reservations, visitor registrations, and role-based administration through a secure and scalable architecture.

Throughout the development of this project, practical experience was gained in frontend development with React, backend development using Node.js and Express.js, database management with MongoDB, RESTful API development, JWT-based authentication, and full-stack application architecture.

The project establishes a strong foundation for future enhancements and demonstrates the application of software engineering principles to solve real-world event management challenges.

References

1. MongoDB Documentation – <https://www.mongodb.com/docs/>
2. React Documentation – <https://react.dev/>
3. Node.js Documentation – <https://nodejs.org/docs/latest/api/>
4. Express.js Documentation – <https://expressjs.com/>
5. Mongoose Documentation – <https://mongoosejs.com/docs/>
6. JWT Documentation – <https://jwt.io/>